

Crawler Superlógica — escolha de tecnologia

Documento de planejamento para decidir se precisamos de um crawler/UI automation e qual ferramenta usar (Puppeteer, Playwright, Cypress, Robot Framework), diante da integração API já existente.

Alvo: Superlógica Imobiliárias

Fase: decisão de arquitetura

Princípio: API first

Login UI: MFA via e-mail

OTP: inbox alternativa

1. Qual problema estamos resolvendo?

Hoje o repositório já extrai cobranças, contratos e taxa adm via API REST (`app_token` + `access_token`). Um crawler só se justifica quando a API não entrega o relatório, a tela ou o export que o negócio precisa.

Já resolvido sem crawler

`/cobrancas` , `/contratos` , `/repasses` → Google Sheets (Apps Script em produção).

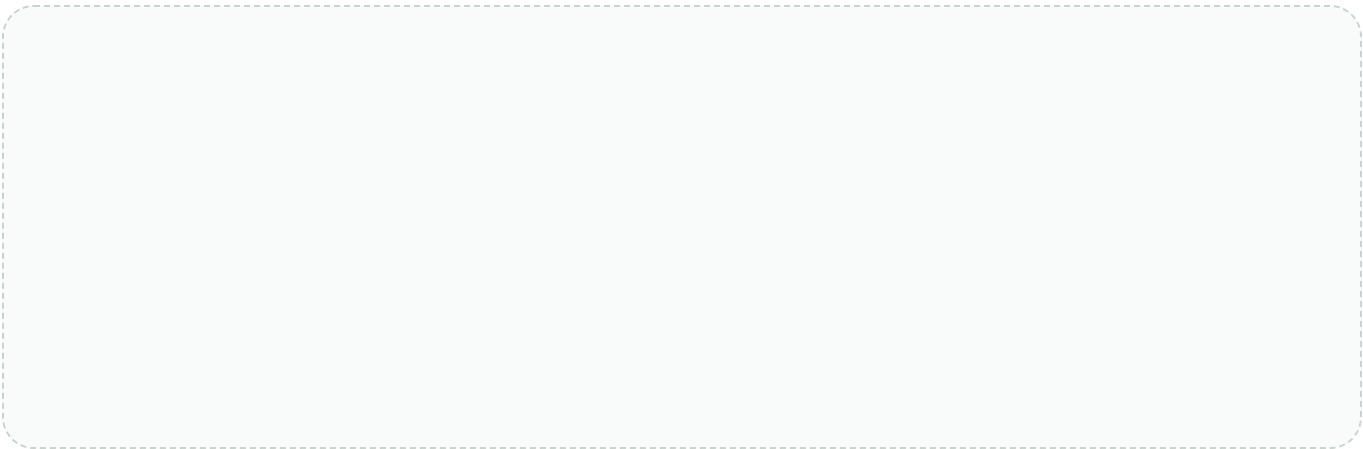
Candidatos a crawler

Relatórios só na UI, export XLSX/PDF sem endpoint, fluxos de aprovação em tela, ou dados que a API lista de forma incompleta.

Antes de escolher ferramenta: listar **quais telas/relatórios** faltam. Sem esse inventário, a decisão de stack fica especulativa. Relatórios de frota/OS (placa, CELESC) em `docs/pdf/*.xlsx` parecem de outro sistema — confirmar se entram no escopo Superlógica.

2. Estratégia recomendada (camadas)

Ordem de preferência — do mais estável ao mais frágil.



CAMADA	QUANDO USAR	FRAGILIDADE
1. API REST	Endpoint documentado ou descoberto; tokens já validados	baixa
2. Network sniff + API "oculta"	UI chama XHR/fetch não listado no OpenAPI — replicar a chamada	média
3. Export automatizado	Relatório gera XLSX/CSV/PDF estável; parse local	média
4. Crawler de DOM	Último recurso: ler tabela/tela, clicar, paginar na UI	alta

A documentação oficial da API Superlógica afirma que a UI é construída sobre a API. Na prática do CEMA já vimos gaps (contratos inativos, listagens incompletas) — mas quase sempre resolvidos ainda na camada 1–2, sem browser.

3. Comparativo das tecnologias (experiência prévia)

Experiência citada: Puppeteer, Cypress, Robot Framework. Incluímos Playwright (sucessor natural do Puppeteer no ecossistema Node).

FERRAMENTA	PROPÓSITO ORIGINAL	BOM PARA CRAWLER?	ENCAIXA NO CEMA?
Playwright	Automação de browser (Chromium/Firefox/WebKit)	excelente	Alinha com Node+TS previsto para Controlle; auto-wait, traces, storageState
Puppeteer	Controle Chrome/Chromium via DevTools	muito bom	Familiar; só Chromium; menos “baterias” que Playwright (multi-browser, codegen)
Cypress	Testes E2E no browser	ruim como crawler	Ótimo para QA; fraco para jobs headless, auth de sessão longa, downloads e cron
Robot Framework	Automação aceitação (keywords + Selenium/Browser)	viável	Stack Python à parte do rumo Node/TS; overhead de keywords para extrair dados
Selenium	Automação clássica multi-browser	viável	Mais boilerplate; drivers; menos DX que Playwright/Puppeteer

CRITÉRIOS DE PONTUAÇÃO (0–5)

CRITÉRIO	PLAYWRIGHT	PUPPETEER	CYPRESS	ROBOT
Extração de dados / ETL	5	5	2	3
Login + sessão persistente	5	4	2	4

CRITÉRIO	PLAYWRIGHT	PUPPETEER	CYPRESS	ROBOT
Download de XLSX/PDF	5	4	2	4
Rodar em cron / CI headless	5	5	3	4
Alinhamento stack CEMA (Node/TS)	5	5	4	1
Manutenção quando UI muda	4	3	3	3
Familiaridade do time	3*	5	5	5
Total	32	31	21	24

*Playwright é o mais próximo do Puppeteer — curva de aprendizado baixa se já houve experiência com Puppeteer.

4. Recomendação (MFA via e-mail alternativo)

MFA continua presente, mas o código/token chega em um **e-mail alternativo** controlado pelo time. O crawler lê essa inbox, extrai o OTP e completa o login — viabilizando job não-atendido.

Confirmado: login tem MFA · **Estratégia:** inbox alternativa recebe a geração do token/OTP · crawler (Playwright + leitor IMAP/API de e-mail) automatiza o desafio. Tokens de API (`app_token` / `access_token`) seguem preferíveis quando o dado já existir na API — o crawler cobre o que a API não entrega.

Dado na API?

Continuar com tokens oficiais (Apps Script / Node). Sem browser.

Dado só na UI?

Playwright: login → poll e-mail OTP → preencher MFA → export/DOM. Job pode rodar em cron.

Ferramenta

Playwright (1ª) ou Puppeteer (alternativa). Cypress/Robot: não.

PEÇAS DO FLUXO MFA POR E-MAIL

PASSO	O QUE FAZ
1. Browser	Abre Superlógica, preenche usuário/senha, dispara envio do MFA
2. Inbox	Poll IMAP (ou Gmail API) na caixa alternativa; filtra remetente/assunto; regex no corpo para extrair o código/token
3. MFA	Cola o OTP na tela e conclui o login
4. Sessão	Persiste <code>storageState</code> para reuso até expirar (evita MFA a cada run)
5. Extração	Navega, exporta XLSX/CSV ou lê DOM; grava Sheets/arquivo

O QUE NÃO RECOMENDAMOS

- **Cypress** — runner de teste; IMAP + OTP + cron ficam tortos.
- **Robot Framework** — só se o time for Python-first; aqui rumo Node/TS.

- **Usar e-mail pessoal de alguém** — inbox deve ser dedicada (conta de serviço / alias).
- **Duplicar na UI** o que já sai de `/cobrancas`, `/repasses`, etc.

Pasta alvo: `integrations/superlogica-crawler/` (Node + TypeScript + Playwright + cliente IMAP/Gmail).

5. Arquitetura alvo (login MFA + e-mail)

Job não-atendido: browser + leitor de inbox alternativa.

```
flowchart TD
    Cron[Cron / Action] --> Job[Job Playwright]
    Job --> Sess{storageState válido?}
    Sess -->|Sim| Nav[Navegar / filtrar]
    Sess -->|Não| Login[Login user/senha]
    Login --> Wait[Aguarda e-mail MFA]
    Mail[(Inbox alternativa  
IMAP / Gmail API)] --> Wait
    Wait --> OTP[Extraí código/token]
    OTP --> MFA[Preenche MFA]
    MFA --> Save[Salva storageState]
    Save --> Nav
    Nav --> Exp{Modo}
    Exp -->|Preferido| DL[Download XLSX/CSV]
    Exp -->|Fallback| DOM[Ler tabelas DOM]
    DL --> Out[Sheets / arquivo / Controlle]
    DOM --> Out
    Job --> Log[Log + screenshot em falha]
```

PEÇA	DECISÃO SUGERIDA
Runtime	Node 20+ / TypeScript
Browser	Playwright Chromium headless
Auth	Credenciais Superlógica em secrets · OTP via e-mail alternativo · <code>storageState.json</code> gitignored para reuso de sessão
Leitor de e-mail	IMAP (<code>imapflow</code>) ou Gmail API com OAuth de conta de serviço. Timeout curto (ex.: 60–120s) + filtro por remetente/assunto recente.
Seletores	Preferir <code>getByRole</code> / texto estável; evitar CSS frágeis de framework
Saída	Sheets, arquivo local ou pipeline Controlle — conforme inventário

PEÇA	DECISÃO SUGERIDA
Observabilidade	Trace Playwright + screenshot + log (sem gravar OTP em log permanente)

6. Riscos específicos do Superlógica

Pontos que mudam a avaliação custo × benefício do crawler.

- **Login com MFA — confirmado** — automatizado via **e-mail alternativo** (poll OTP). Risco: atraso/entrega do e-mail, mudança de template, bloqueio de IMAP.
- **Segurança da inbox** — conta dedicada, senha de app / OAuth, sem compartilhar e-mail pessoal; OTP não deve ir para logs permanentes.
- **SPA e seletores** — mudanças de front quebram scripts; exports e XHR quebram menos.
- **Rate limit / sessão** — reusar `storageState` para não disparar MFA a cada execução.
- **Compliance** — automação de UI + leitura de e-mail; tokens oficiais de API seguem preferíveis quando existirem.
- **Duplicidade** — não reimplementar em crawler o que o Apps Script já cobre via API.

7. Checklist de decisão (para fechar juntos)

Respostas a estas perguntas fecham a stack.

#	PERGUNTA	IMPACTO
1	Quais relatórios/telas exatamente faltam à API atual?	respondido Despesas (financeiro) — flstatus=2&flcreditodebito=2
2	Esses dados aparecem em Network (XHR) ao abrir a tela?	Se sim → camada 2 (API oculta), sem Playwright
3	Existe botão de export XLSX/CSV/PDF estável?	Se sim → Playwright só para download + parse
4	Login é user/senha simples ou tem MFA?	MFA + OTP no e-mail alternativo → crawler em cron viável (browser + IMAP)
4b	Provedor da inbox (Gmail, Outlook, outro)? IMAP ou API?	Define cliente de e-mail e secrets
5	Frequência: sob demanda, diário, a cada hora?	Define hosting (Action, VM, local)
6	Destino: Sheets, Controlle, arquivo, ambos?	Define pasta e contrato de saída
7	Preferência do time: Playwright novo ou Puppeteer conhecido?	Playwright (escolha 1)

8. Próximos passos sugeridos

1. Inventariar telas/relatórios desejados (lista curta, com print ou URL).
2. Para cada item: DevTools → Network — se houver XHR, preferir API com tokens.
3. Confirmar provedor da inbox alternativa (Gmail/Outlook) e acesso (IMAP + senha de app ou OAuth).
4. Spike: Playwright login → poll e-mail → OTP → 1 export; medir tempo até o e-mail chegar e estabilidade do regex.
5. Persistir `storageState` e só refazer MFA quando a sessão cair.

Não versionar senha, OTP nem `storageState`. Inbox deve ser conta/alias dedicada ao robô.

9. Registro de decisão

Decisões fechadas em 04/08/2026 — implementação em andamento.

CAMPO	VALOR
Data	04/08/2026
URL alvo (v1)	/clients/financeiro/despesas/index?flstatus=2&flcreditodebito=2 tenant: testejunior.superlogica.net
Login UI	MFA confirmado · OTP via e-mail alternativo
Estratégia MFA	Crawler lê inbox → extrai token/código → completa login automaticamente
Camada escolhida	UI crawler (camada 3–4) · API paralela se Network revelar XHR útil
Ferramenta	Playwright + IMAP (escolha 1)
Pasta no monorepo	integrations/superlogica-crawler/
Fora de escopo	Cypress · Robot · Puppeteer · usar e-mail pessoal · duplicar cobranças/repasses já em Apps Script